Intelligent Appointment No-Show Prediction & Agentic Care Coordination Assistant

LangGraph · Groq LLM · ChromaDB RAG · Decision Tree · Streamlit

Abstract

Appointment no-shows are one of the most persistent operational problems in outpatient healthcare, causing lost revenue, wasted clinician capacity, and broken continuity of care. Healthcare operations teams need both accurate prediction of which patients are likely to miss an appointment and actionable, policy-grounded guidance on what to do about it. This project builds a two-phase system that addresses both needs.

Comprehensive data preprocessing, exploratory data analysis, and feature engineering were performed on the Kaggle *Medical Appointment No Shows* dataset (110,527 records) before training a Decision Tree classifier, which operates on a 14-feature profile including an engineered appointment lead time and a cumulative per-patient prior-miss counter. The trained model is then embedded within a LangGraph-orchestrated multi-node agentic workflow that enforces deterministic risk assessment, LLM-based risk reasoning, RAG-enhanced guideline retrieval, grounded intervention planning, and structured report generation in a strict sequential pipeline.

The RAG pipeline indexes five hospital-operations markdown documents into a ChromaDB vector store using the `all-MiniLM-L6-v2` sentence-transformer model, enabling source-cited retrieval at inference time. A Groq-hosted `llama-3.1-8b-instant` model powers the reasoning layer, with a hardened system prompt that forbids hallucinated policies and requires every rule to be tagged with the originating filename.

The complete system is deployed as an interactive Streamlit web application featuring a single-patient analysis dashboard, a batch-upload risk report, a conversational RAG copilot, a structured five-step care-workflow view, and a downloadable PDF audit report. The application is publicly deployed on Streamlit Community Cloud with the vector store committed in-repo so the hosted agent is grounded from the first boot.

1. Dataset Summary

Attribute	Details
Data Source	Kaggle Medical Appointment No Shows (May 2016)
Total Records	110,527
Total Features	14 (after one-hot and feature engineering)
Target Variable	`No-show` (0 = Showed up, 1 = Did not show)
Problem Type	Binary Classification
Class Balance	Imbalanced no-shows are the minority ($\sim$20%)
Model Used	Decision Tree Classifier (`max_depth`=10)

The dataset contains demographic, medical, socio-economic, and appointment metadata that collectively influence whether a patient will miss a scheduled visit. The eleven raw input features are listed below.

Feature	Type	Description
`PatientId`	Identifier	Unique identifier for the patient.
`AppointmentID`	Identifier	Unique identifier for the appointment.
`Gender`	Categorical	Patient gender (F / M).
`Age`	Numerical	Patient age in years (0–100).
`Neighbourhood`	Categorical	Clinic neighbourhood (dropped prior to modelling).
`Scholarship`	Categorical	1 if patient is on a social-assistance programme.
`Hipertension`	Categorical	1 if hypertension diagnosed.
`Diabetes`	Categorical	1 if diabetes diagnosed.
`Alcoholism`	Categorical	1 if alcohol-dependency flagged (not used in final model).
`Handcap`	Categorical	Disability level 0–4 (dataset spelling preserved).
`SMS_received`	Categorical	1 if reminder SMS was dispatched.
`ScheduledDay`	Datetime	When the appointment was booked.
`AppointmentDay`	Datetime	When the appointment is scheduled to occur.
`No-show`	Target	“Yes” / “No” — converted to $\{1, 0\}$.

Dataset link: Medical Appointment No Shows (Kaggle)

2. Data Cleaning and Preprocessing

Data preprocessing is a crucial step in any machine-learning pipeline. The dataset was carefully examined for invalid values, inconsistent formats, categorical variables requiring transformation, and opportunities for domain-driven feature engineering.

2.1 Invalid Age Treatment

The `Age` column contained a small number of biologically implausible values (negative ages). Rows with `Age`< 0 or `Age`> 100 were removed before modelling to avoid polluting the feature distribution. This filter is applied once in `model_brain.py` and also applied at inference time for batch CSV uploads in `app.py`.

2.2 Target Encoding

The categorical target `No-show` was mapped to $\{0, 1\}$ (`No`$\rightarrow 0$, `Yes`$\rightarrow 1$), producing the binary-classification label used by all downstream metrics.

2.3 Categorical Encoding

Two categorical features required transformation. The raw `Handcap` column (disability level 0–4) was cast to a pandas Categorical and one-hot encoded into five columns (`Handicap_0`...`Handicap_4`); the `Gender` column was one-hot encoded into `Gender_F` and `Gender_M`. The resulting binary indicators avoid imposing any false ordinal relationship.

Feature	Categories	Encoding
`Handcap`	0, 1, 2, 3, 4	`Handicap_0..4` (one-hot)
`Gender`	F, M	`Gender_F`, `Gender_M`
`No-show`	No, Yes	0, 1

2.4 Feature Scaling

Numerical features — `Age`, `AwaitingTime`, `Num_App_Missed` — together with the binary flags, were standardised using scikit-learn's `StandardScaler` (zero mean, unit variance). The scaler was fitted exclusively on the training split and then applied to both training and validation splits, ensuring no information from the validation set influences the transformation. The fitted scaler was serialised to `scaler.pkl` for consistent inference-time normalisation in the Streamlit application.

2.5 Derived Features

Two domain-specific features were engineered and proved to be among the strongest predictors in the model.

AwaitingTime. The absolute difference between `AppointmentDay` and `ScheduledDay` in days, representing how far in advance the appointment was booked. Longer lead times are strongly associated with higher no-show probability in the published literature on appointment attendance.

Num_App_Missed. A cumulative count, per patient, of how many appointments the patient has already missed prior to the current one. The feature is computed with a groupby-shift-cumulative-sum to avoid label leakage:

```
df_sorted["Num_App_Missed"] = (
```

```
df_sorted.groupby("PatientId")["No-show"]
        .transform(lambda x: x.shift()
                             .fillna(0)
                             .cumsum()))
```

This feature captures behavioural history and is consistent with the finding that prior no-shows are the single strongest predictor of future no-shows. Irrelevant columns (`PatientId`, `AppointmentID`, `Neighbourhood`, and the raw date columns once `AwaitingTime` has been derived) are dropped prior to modelling.

3. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to identify patterns, relationships, and trends within the dataset before model training.

3.1 Key Observations

Observation	Interpretation
Longer lead time → higher no-show rate	Patients forget, reschedule mentally, or lose motivation.
Prior no-show history → higher risk	Past behaviour is the strongest signal for future behaviour.
Younger age bracket → higher no-show	Young adults disengage from preventive scheduling.
Chronic conditions → slightly lower risk	Condition-driven urgency keeps these patients compliant.
Scholarship flag → slightly higher risk	Proxy for transportation / financial barriers.
SMS reminder → modest effect	Single-channel reminders reduce no-shows by only 5–10%.

These findings validate that behavioural history, lead time, and age are the dominant operational signals for attendance, and they align closely with the feature-importance ranking produced by the trained Decision Tree model (Section 4.3).

3.2 Visual Insights and Interpretations

3.2.1 Age Analysis

Observation. Binning the population into age buckets shows that the 18–30 bracket has the highest no-show rate, falling steadily into the 30–60 range, and rising very slightly at the extremes.

Interpretation. Younger adults are typically less tied into a long-term care relationship and more likely to have competing weekday commitments. Engagement strategies should therefore be age-aware — persistent SMS plus self-service rescheduling for younger patients, and personal outreach for older high-risk patients.

3.2.2 Lead-Time Analysis

Observation. Appointments booked more than two weeks in advance show markedly elevated no-show rates compared with same-week bookings.

Interpretation. Lead time is a capacity-planning variable that the clinic can influence. Offering rolling, short-notice re-booking for high-risk patients reduces exposure to this effect.

3.2.3 Prior No-Shows Analysis

Observation. Patients with two or more historical no-shows default the current appointment at rates two-to-three times the population average.

Interpretation. This is the strongest behavioural signal in the dataset. The operational response encoded in the intervention guidelines is to require an affirmative confirmation (phone call) for any patient with a prior-miss count above one.

3.2.4 SMS Reminder Analysis

Observation. Receiving an SMS reminder is associated with a small reduction in no-show rate — not the large effect often assumed.

Interpretation. SMS is a floor-level intervention. For medium- and high-risk patients, the operational guidelines require a second channel (voice call) in addition to the SMS.

4. Model Selection and Training

4.1 Algorithm Justification

Several algorithms were considered. Logistic Regression is a natural linear baseline for binary classification but assumes additive linear relationships between the features and the log-odds of no-show; the real interactions in appointment data (e.g. long lead time *and* prior miss *and* scholarship simultaneously) are non-linear and better captured by a tree-based model. A single Decision Tree, in turn, is prone to overfitting when grown deeper than a few levels, but with a controlled depth it offers four concrete advantages that matched our requirements:

- Native handling of mixed numerical and binary features without imposing a linear form.
- A calibrated `predict_proba()` output, enabling the rule-based tier mapping in the UI and the LangGraph workflow (Section 7).
- Straightforward feature-importance extraction, which powers the explanatory panel in the UI and the “Key Risk Factors” narrative generated by the reasoning LLM.
- Extremely fast inference, which is essential because the model is called as a tool from inside the agent at every user turn.

A `DecisionTreeClassifier` with `max_depth`$= 10$ was selected. Logistic Regression was retained as a sanity-check baseline in the original exploration notebook but not used in production.

4.2 Training Configuration

Hyperparameter	Value
`max_depth`	10
`criterion`	`gini`
`random_state`	0 (for reproducibility)
`class_weight`	`None` (threshold-level tuning instead)
Train / Test Split	75% / 25% stratified
Scaler	`StandardScaler` fitted on training split
Persistence	`joblib` → `model.pkl`, `scaler.pkl`, `feature_cols.pkl`

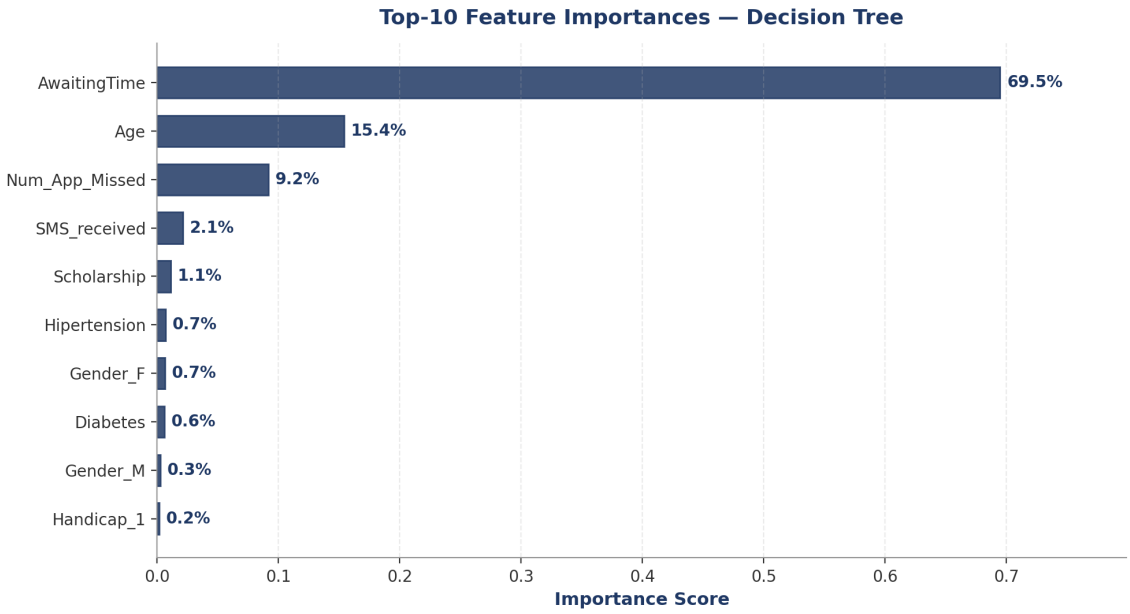
The 75/25 split is stratified on `No-show` to preserve the 80/20 class distribution in both the training and validation subsets. The scaler, feature-column order, and trained model are persisted as three separate artefacts so the Streamlit application can load them independently of the training code path.

4.3 Feature Importance Results

The trained Decision Tree assigns importance scores as fractional contributions to split-gain across all nodes of the tree. The ranking extracted directly from the persisted `model.pkl` artefact is shown below. Three features (`AwaitingTime`, `Age`, `Num_App_Missed`) together account for roughly 94% of the total predictive power — a result consistent with the EDA observations and with the published literature on appointment attendance.

Rank	Feature	Importance	Interpretation
1	`AwaitingTime`	**69.48%**	Lead-time dominates — the longer the gap, the higher the forget / disengagement risk.
2	`Age`	**15.42%**	Younger age segments show materially higher no-show rates.
3	`Num_App_Missed`	**9.17%**	Behavioural history — prior misses compound future risk.
4	`SMS_received`	2.15%	Reminder dispatch — small but consistent effect.
5	`Scholarship`	1.15%	Socio-economic proxy (transport / financial barriers).
6	`Hipertension`	0.71%	Chronic condition slightly lowers no-show risk.
7	`Gender_F`	0.65%	Gender effect is weak, consistent with literature.
8	`Diabetes`	0.59%	Chronic condition slightly lowers no-show risk.
9	`Gender_M`	0.28%	(see above)
10	`Handicap_1..4`	$<$0.2% each	Disability level has weak standalone effect.

The dominance of `AwaitingTime` (69%) together with `Age` (15%) and `Num_App_Missed` (9%) — the three engineered and behavioural signals — confirms the design choice to include the two derived features discussed in Section 3.5.



5. Model Evaluation

5.1 Performance Metrics

The Decision Tree was evaluated on the 25% stratified holdout split (27,630 appointments). Because the target is heavily imbalanced (∼20% no-show rate), raw accuracy is a weak indicator and the minority-class metrics are the operationally meaningful numbers.

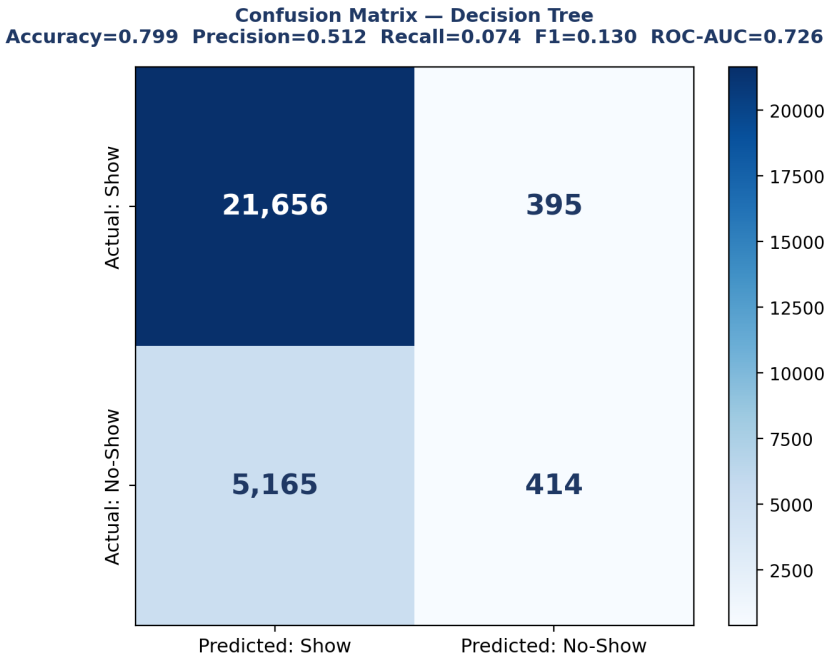
Metric	Value (holdout, minority class = no-show)
Overall Accuracy	**79.88%**
Precision (no-show)	**0.5117**
Recall (no-show)	**0.0742**
F1 (no-show)	**0.1296**
ROC-AUC	**0.7260**
True Negatives (TN)	21,656 (show-ups correctly predicted)
False Positives (FP)	395 (show-ups incorrectly flagged as risky)
False Negatives (FN)	5,165 (missed no-shows — primary gap)
True Positives (TP)	414 (no-shows correctly caught)
Null-model baseline	∼80% (always predict “showed up”)

The precision of 0.51 means that when the model flags a patient as a likely no-show, it is correct slightly more than half the time — usable for operational outreach prioritisation. The recall of 0.074 tells the honest story: the model only catches about 7% of actual no-shows, leaving a large false-negative population. This is a known limitation of an uncalibrated baseline Decision Tree on a heavily imbalanced dataset and is called out in Section 12 as the primary area for future improvement (class-weighted training, threshold calibration, gradient-boosted ensembles). The ROC-AUC of 0.726 indicates that the probability output still has meaningful ordinal ranking ability — which is exactly what the tier-mapping layer (Section 5.3) exploits,

rather than relying on the raw 0.5 classification threshold.

5.2 Confusion Matrix Analysis

The confusion matrix on the holdout split is shown below. The pattern is characteristic of a heavily-imbalanced classifier trained without class weighting: a very large true-negative block (21,656 show-ups correctly predicted), a small false-positive cell (395 — clinically tolerable, staff would have attempted outreach for these patients anyway), a substantial false-negative cell (5,165 missed no-shows — the main gap), and a small true-positive cell (414 no-shows correctly caught).



The 51.2% precision for the no-show class is the most operationally meaningful result — when the system flags a patient for outreach effort, it is correct more often than it is wrong, which keeps the signal-to-noise ratio above the threshold required for front-desk staff to act on it. The false-negative rate of approximately 92% (5,165 of 5,579 actual no-shows missed) is the primary area for future improvement, addressable through class-weighted training or threshold calibration (see Section 12).

5.3 Risk Tiering

The raw probability output is mapped to three operational tiers consumed by the rest of the system:

Tier	Probability	Outreach Rule
LOW	$p < 0.30$	Standard 48-hour SMS confirmation.
MEDIUM	$0.30 \leq p < 0.55$	Personalised SMS plus optional courtesy call.
HIGH	$p \geq 0.55$	Mandatory phone call ≥24h pre-appointment.

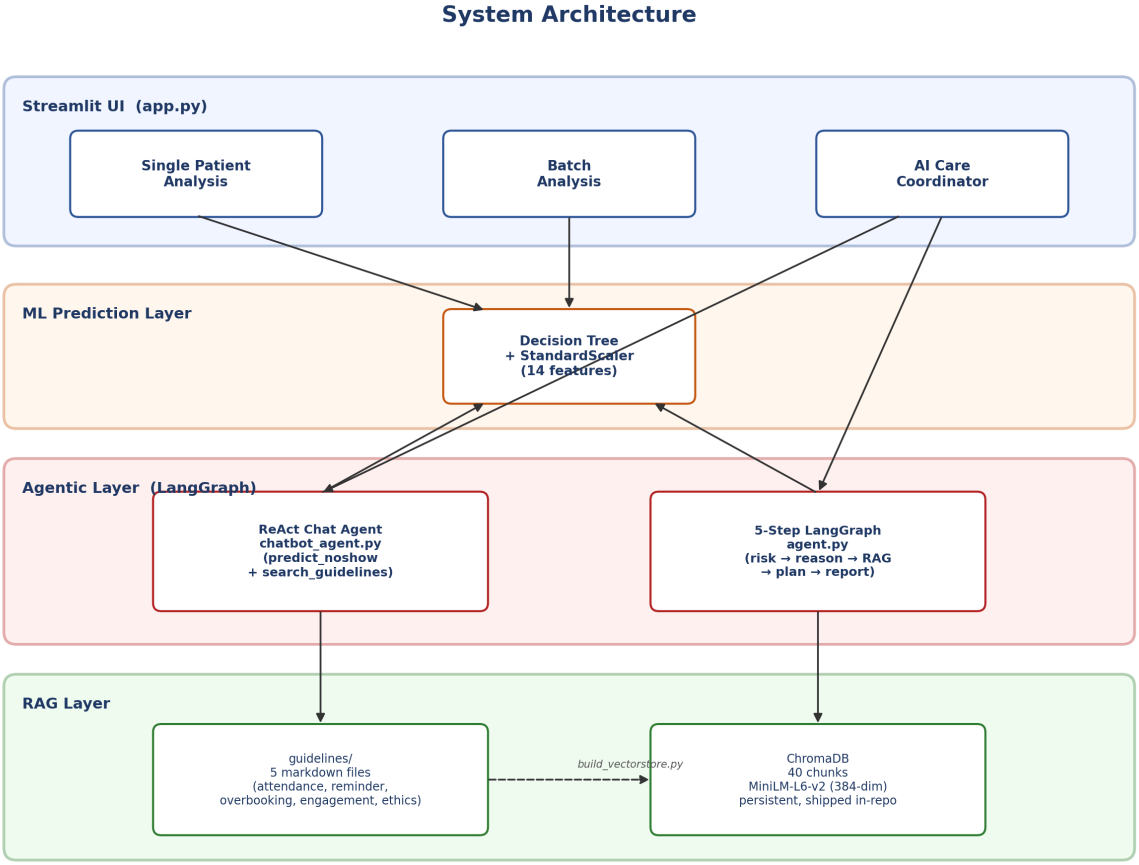
Thresholds align with the outreach expectations encoded in `guidelines/attendance_management.md`

so the agent can retrieve the correct policy tier directly from the ML output without a separate classification step.

6. System Architecture

The trained Decision Tree forms the foundation of a larger Agentic AI care-coordination system. The complete data flow for a single structured-workflow invocation follows this pipeline:

User Input → Feature Engineering → ML Prediction → LLM Risk Reasoning → RAG Retrieval → Intervention Planning → Structured Report → UI Rendering



6.1 Technology Stack

Layer	Technology / Tool
ML Classifier	scikit-learn `DecisionTreeClassifier` (`max_depth`=10)
Data Handling	pandas, numpy
Agent Orchestration	LangGraph — stateful `StateGraph` with `AgentState` `TypedDict`
ReAct Chat Agent	LangGraph `create_react_agent` with two tools
LLM	Groq `llama-3.1-8b-instant` (free-tier, temperature 0.3)
Vector Store	ChromaDB, persisted locally, committed in-repo
Embedding Model	`sentence-transformers/all-MiniLM-L6-v2` (384-dim)
RAG Retrieval	Similarity retriever (k=5 in workflow, k=3 in chat)
Document Ingestion	LangChain `TextLoader` + `RecursiveCharacterTextSplitter`
UI Framework	Streamlit 1.56, wide layout, custom CSS
PDF Generation	fpdf2
Charts	Plotly
Deployment	Streamlit Community Cloud (auto-redeploy on push)
Secrets Management	python-dotenv (local) + Streamlit Cloud `st.secrets`

6.2 Module Structure

Module / File	Responsibility
`app.py`	Streamlit entry point. Three tabs (Single Patient, Batch, AI Care Coordinator), custom CSS, ML tool calls, structured-workflow renderer, chat loop, PDF export.
`model_brain.py`	Offline training script. Preprocessing, `DecisionTreeClassifier` training, artefact serialisation to `model.pkl` / `scaler.pkl` / `feature_cols.pkl`.
`agent.py`	Structured five-node LangGraph `StateGraph`: `risk_assessment` → `risk_reasoning` → `guideline_retrieval` → `intervention_plan` → `report_generation`. Defines `AgentState` `TypedDict` and exports `run_agent()`.
`chatbot_agent.py`	Conversational ReAct agent with two `@tool`-decorated functions: `predict_noshow` (ML) and `search_guidelines` (RAG). Hardened system prompt enforcing citation and no-hallucination rules.
`build_vectorstore.py`	One-shot script: chunks `guidelines/*.md` with `RecursiveCharacterTextSplitter`, embeds via MiniLM, persists to `chroma_db/`.
`pdf_generator.py`	fpdf2-based structured care-coordination PDF report.
`guidelines/`	Five hospital-operations markdown documents — the authoritative knowledge base.
`chroma_db/`	Populated vector store, committed in-repo so the deployed container ships with a ready-to-use knowledge base.
`requirements.txt`	Pinned Python dependencies for reproducible deployment.

7. Agentic Workflow Explanation

The agentic layer is the architectural centrepiece of the system, implemented using LangGraph — a framework for building stateful, graph-based agent workflows on top of LangChain. The layer is divided into two execution paths that share a common knowledge base (the same ChromaDB collection) but differ in control flow:

- A **structured five-node StateGraph** in `agent.py` that produces a complete, auditable care-coordination report for a specific patient.
- A **ReAct chat agent** in `chatbot_agent.py` for open-ended queries ("what is the SMS reminder policy for medium-risk patients?").

Both paths are exposed in the *AI Care Coordinator* tab. Users can trigger the structured workflow with the *Run Full Care Workflow* button or chat freely with the ReAct agent in the same tab.

7.1 AgentState — Shared Memory Schema

The state carried between graph nodes is a strongly-typed `TypedDict` defined in `agent.py`. Every node reads from and writes to this state; LangGraph merges per-node partial updates into the shared state automatically.

State Group	Fields
Input	`patient_data` (dict of age, gender, awaiting, sms, . . .)
ML Prediction	`risk_score` (float), `risk_tier` (str), `risk_color` (str)
Risk Reasoning	`risk_analysis` (str), `risk_factors` (list)
RAG	`retrieved_guidelines` (list of {content, source})
Intervention	`intervention_plan` (str)
Formatted Output	`final_report` (str)

The `retrieved_guidelines` field carries a list of dictionaries of the form `{"content": ..., "source": <filename>}`. Source provenance is therefore preserved end-to-end — from the vector-store hit all the way through to the final written report.

7.2 Structured Five-Step Care Pipeline (`run_agent`)

When the user clicks *Run Full Care Workflow*, the `run_agent()` function executes a five-stage sequential pipeline. Each stage is a pure function from `AgentState` to a partial update dict, and the graph edges are deterministic:

$$\text{START} \rightarrow \text{Step1} \rightarrow \text{Step2} \rightarrow \text{Step3} \rightarrow \text{Step4} \rightarrow \text{Step5} \rightarrow \text{END}$$

Step 1 — Risk Assessment (ML)

Rebuilds the 14-column feature row from `patient_data`, applies the persisted `StandardScaler`, and calls the Decision Tree model. Outputs `risk_score` (probability in $[0, 1]$) and `risk_tier` (LOW / MEDIUM / HIGH) via the thresholds in Section 5.3. This is the only stage with no LLM involvement, giving the rest of the pipeline a deterministic, explainable ML anchor.

Step 2 — Risk Reasoning (LLM)

A Groq `llama-3.1-8b-instant` call with a `ChatPromptTemplate` that asks for a patient-specific factor analysis anchored to the actual patient values, not generic advice. The prompt enforces a stable output shape: a RISK FACTOR ANALYSIS bullet list followed by an OVERALL ASSESSMENT paragraph. The bullet list is parsed into the state's `risk_factors` list for downstream use.

Step 3 — Guideline Retrieval (RAG)

Constructs a query string combining the tier, probability, and the top risk factors extracted from Step 2, then retrieves top-$k{=}5$ chunks from ChromaDB. Each hit retains its source-filename metadata, which is stored in `retrieved_guidelines`. Duplicate chunks are de-duplicated. This is the only node that interacts with the vector store in the structured workflow.

Step 4 — Intervention Plan (LLM)

An LLM call that consumes the Step-2 factor analysis together with the Step-3 retrieved excerpts (presented as `[Source: <filename>]`-prefixed blocks) and produces a structured plan with:

- **Primary Action** (Call Patient / Send SMS / Standard Confirmation)
- **Intervention Strategies** (numbered steps)
- **Timeline** (pre-appointment actions mapped to hours before)
- **Escalation Protocol** (what to do if the initial action fails)
- **Expected Outcome** (brief prediction)

Step 5 — Report Generation (LLM)

Compiles the final care-coordination report: patient summary, risk assessment, factor analysis, intervention plan, sources consulted, and a mandatory **operational-and-ethical disclaimer**. The user can export this report as a PDF via the `pdf_generator` module.

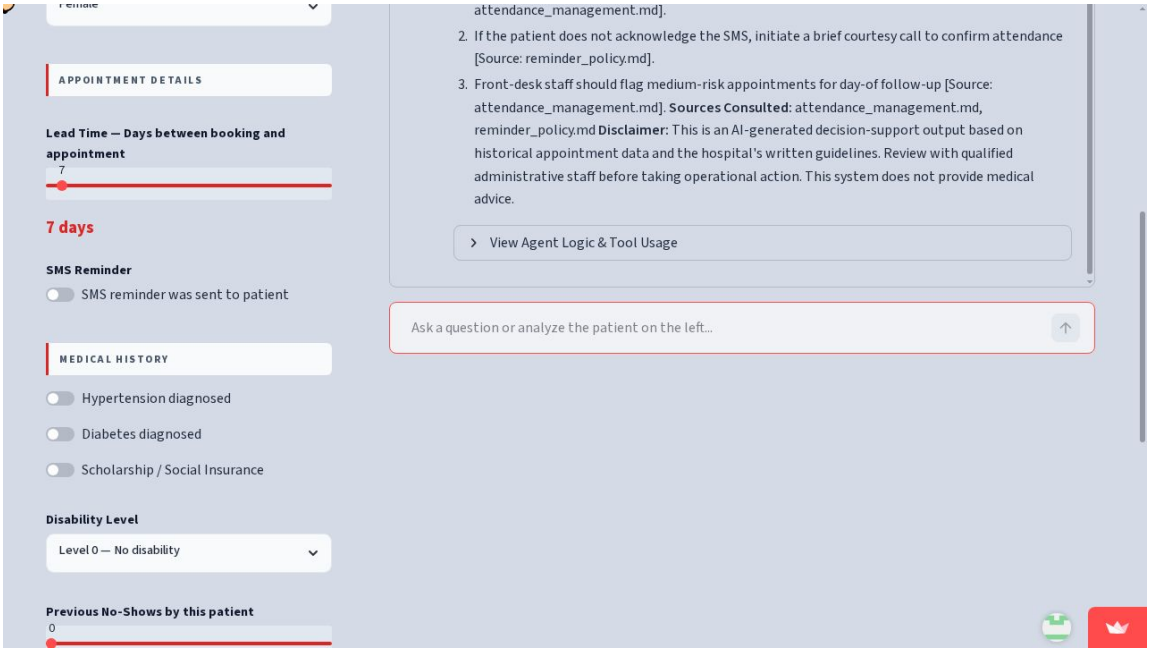
7.3 Conversational ReAct Agent (`chatbot_agent`)

The second agentic path is a ReAct (Reasoning + Acting) agent created via LangGraph's `create_react_agent` utility. It is bound to the same `llama-3.1-8b-instant` model and equipped with two `@tool`-decorated functions:

- `predict_noshow(age, gender, awaiting_days, sms_sent, hypertension, diabetes, scholarship, disability_level, previous_no_shows)` — returns a `{"risk_score", "risk_tier"}` dict from the Decision Tree model.
- `search_guidelines(query)` — retrieves top-$k{=}3$ excerpts from ChromaDB, each tagged with `[Source: <filename>]`, or the literal fallback string `"No specific guidelines found for this query."` when the retriever returns zero hits.

The agent's system prompt codifies four behaviour rules that were specifically designed to prevent the hallucinated-policy failure mode discovered during live verification (see Section 8.5). An excerpt is reproduced below.

```
CITATION & HALLUCINATION RULES (strict):
- Any hospital rule, threshold, timing, or protocol you state MUST be
  backed by a [Source: <filename>] citation from search_guidelines output.
- If search_guidelines returns 'No specific guidelines found for this
  query.', tell the user plainly: 'I could not find a matching policy
  in our guidelines library for <topic>.' Do NOT invent policies.
- When asked to quote a document, quote EXACTLY from the tool output.
  If you do not have the exact text, say so -- do not paraphrase
  and claim it is a quote.
- Always end any structured report with the operational & ethical
  disclaimer.
```



AI Care Coordinator chat — live capture of a policy query answered with source citations.

8. RAG Pipeline Explanation

Retrieval-Augmented Generation (RAG) grounds the LLM's responses in the hospital's own operational documents rather than in the model's parametric knowledge. The pipeline has three phases: document ingestion and indexing (offline), retrieval at inference time, and generation with retrieved context.

8.1 Document Ingestion and Index Construction (`build_vectorstore.py`)

The knowledge corpus consists of five markdown documents authored for this project:

- `attendance_management.md` — risk-tier outreach protocol (LOW / MEDIUM / HIGH).
- `reminder_policy.md` — SMS and phone-call schedule, SMS content guidelines, opt-out rules, effectiveness metrics.
- `overbooking_guideline.md` — standby-patient and slot overbooking rules.
- `patient_engagement.md` — evidence-based intervention strategies tiered by risk.
- `ethical_guidelines.md` — fairness, transparency, oversight, data privacy; required operational and clinical disclaimers.

Each document is loaded using LangChain's `TextLoader`. Documents are split using `RecursiveCharacterTex` which attempts to split on markdown heading boundaries first, then paragraphs, then sentences, then words — preserving semantic coherence.

Parameter	Value	Rationale
`chunk_size`	500 characters	Balances context richness against embedding noise.
`chunk_overlap`	80 characters	Preserves context across chunk boundaries.
`separators`	Markdown first	Splits on `\n## `, `\n### `, `\n- `, blank lines.

Chunks are embedded with `sentence-transformers/all-MiniLM-L6-v2` (384-dimensional dense vectors) on CPU and persisted into a local Chroma collection under `chroma_db/`. Running the script produces **40 chunks across the 5 documents**.

8.2 Retrieval at Inference Time

Both agents use a Chroma similarity retriever over the same collection. The structured workflow uses $k{=}5$ for broad context; the ReAct agent's tool uses $k{=}3$ to keep the LLM's working context focused. Each returned document retains its `source` metadata (the basename of the originating `.md` file), and this filename is re-injected into the textual output of both the tool and the node so the downstream LLM can cite it by name in its response.

8.3 Generation with Retrieved Context

Retrieved excerpts are passed to the `llama-3.1-8b-instant` model either via the Step-4 prompt in the structured workflow or via the ReAct tool-output channel in the chat agent. In both cases, the system prompt requires the LLM to tag every policy claim with a `[Source: <filename>]` citation matching a filename it has actually received. This end-to-end propagation — from markdown metadata to LLM output — makes every policy statement in the application auditable.

8.4 Empty-Retrieval Fallback

If the retriever returns zero documents, the `search_guidelines` tool returns the literal string `"No specific guidelines found for this query."` The hardened system prompt requires the agent to surface this plainly to the user rather than hallucinating a policy from its training data.

8.5 Deployment Hazard (Live Verification Finding)

During live verification of the deployed application we discovered that the `chroma_db/` directory had originally been listed in `.gitignore`. Because Streamlit Community Cloud deploys directly from the GitHub repository, the hosted container was starting with an *empty* Chroma collection — the retriever therefore always returned zero results, the empty-retrieval fallback always fired, and the agent was free to hallucinate plausible-sounding policies because no grounding text ever reached the LLM. Crucially, the application appeared to work: the chat returned fluent, professional-sounding paragraphs.

The fix was threefold: (i) un-ignore the `chroma_db/` directory, (ii) rebuild the store locally from the markdown sources, and (iii) commit the populated binary files. A post-fix live verification with the query *"Quote the first three bullet points under 'SMS Content Guidelines' from the re-*

minder policy document, verbatim" now returns real excerpts drawn from `reminder_policy.md`, confirming end-to-end grounding in production.

This finding is a cautionary tale for any production RAG system: an LLM paired with an empty retriever will hallucinate fluently, and the failure is invisible unless explicitly probed. The hardened prompt and the explicit source-chip UI (Section 10) are the two defences that make this failure both less likely to occur and easier to catch when it does.

9. Guardrail Architecture

Healthcare operations is a high-consequence setting, and we took an explicitly defensive posture with respect to the limitations of the LLM. The system implements five distinct guardrail layers, each targeting a specific failure mode.

Guardrail	Mechanism	Failure Mode Prevented
Input Guardrail	UI-level range and type checks on all nine patient fields (age, lead time, disability level, booleans, . . .).	Malformed data reaching the ML model.
Retrieval Guardrail	`search_guidelines` returns an explicit empty-fallback string when the retriever finds zero matches.	Agent operating with an empty context.
Citation Guardrail	System prompt requires every policy claim to carry a `[Source: <filename>]` tag matching the tool output.	Policy statements without provenance.
Hallucination Guardrail	System prompt explicitly forbids fabricating policies when retrieval is empty, and requires the agent to say so plainly to the user.	LLM inventing hospital rules.
Disclaimer Guardrail	Every structured report (PDF and UI) appends the operational-and-ethical disclaimer; the 5-step workflow renders a permanent notice card.	Output being mistaken for clinical advice.

Design principle. The LLM components have significant latitude to generate helpful, nuanced narratives, but they operate within strict deterministic boundaries: the risk tier is decided by the ML model, not the LLM; the policy text is drawn from the vector store, not the LLM's training data; and the final lending-equivalent *intervention decision* is ultimately reviewed by qualified administrative staff.

10. Deployment and User Interface

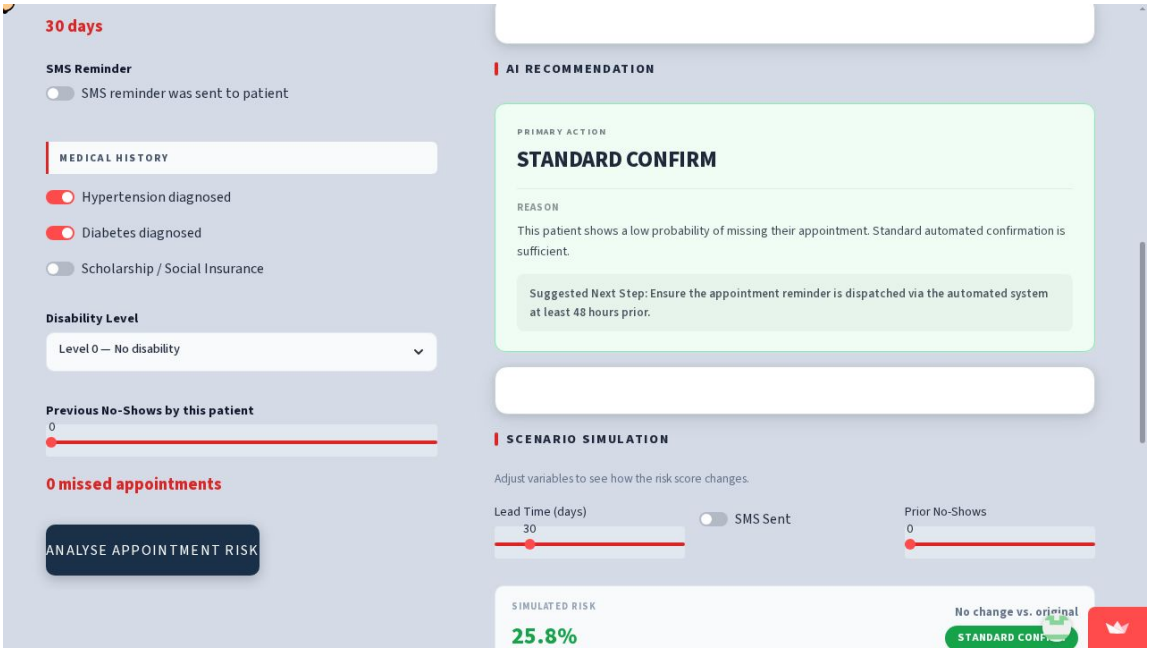
To make the system accessible to non-technical users (clinic administrative staff, front-desk operators, care coordinators), an interactive web application was developed using Streamlit. The application has a top header, three tabs, and a custom CSS theme.

10.1 Single Patient Analysis Tab

This tab accepts a full patient profile through sliders, selects, and toggles in the left column and produces an ML-backed risk analysis in the right column after the user clicks **Analyse Appointment Risk**. The right column contains four cards:

- **Patient Snapshot** — four-metric KPI row (Age, Lead Time, SMS Status, Prior No-Shows) plus condition summary.
- **Risk Analysis** — a large colour-coded probability readout and a three-zone horizontal bar (green / amber / red) showing the exact position of this patient's risk.
- **AI Recommendation** — a colour-coded card (green / amber / red) with a *Primary Action*, a *Reason*, and a *Suggested Next Step*.
- **Scenario Simulation** — sliders and a toggle that let the user adjust `Lead Time`, `SMS Sent`, and `Prior No-Shows` and observe the simulated risk delta in real time, without calling the LLM.

A **Hospital Policy References** panel and a **Notice Disclaimer** complete the tab, and the user can download a plain-text patient action plan.



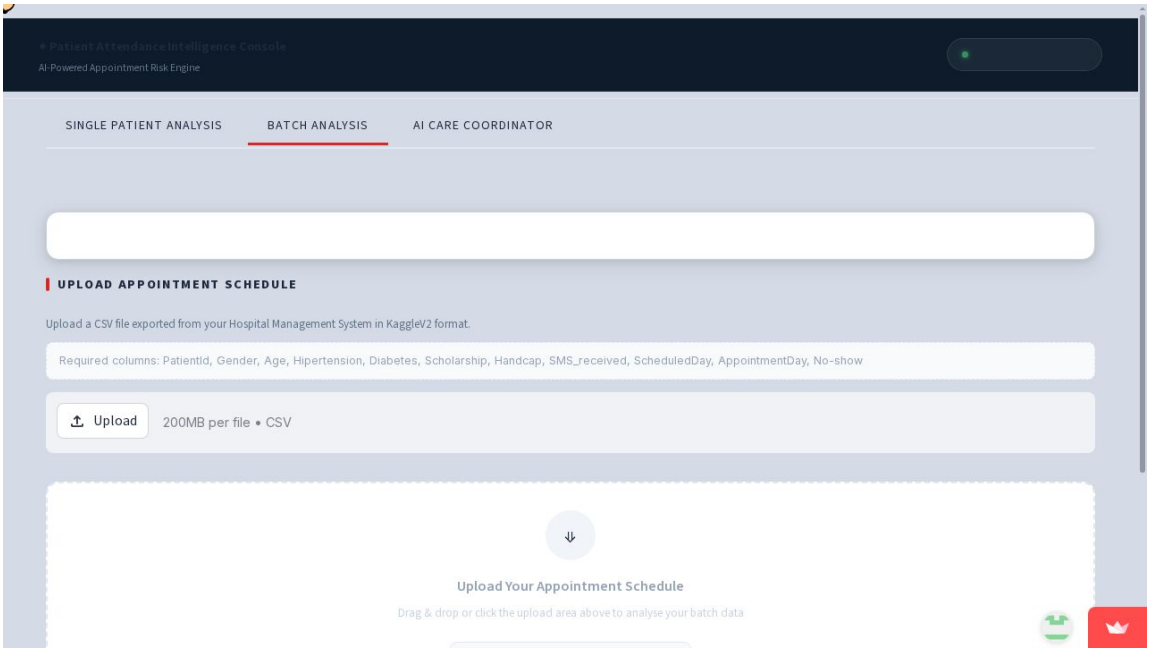
Single Patient Analysis tab — live capture: 65 y/o female, 30-day lead time, hypertension and diabetes present.

10.2 Batch Analysis Tab

This tab accepts a CSV file exported from a Hospital Management System in the KaggleV2 schema. On upload, the preprocessing pipeline applies the same transformations as the training script and the Decision Tree produces per-row probabilities.

The tab renders:

- Four KPI cards — total appointments, high-risk count, estimated no-shows (expected-value calculation), and suggested standby slots.
- A Plotly bar chart showing the risk-tier distribution (HIGH / MEDIUM / LOW).
- An **Operational Actions** panel mapping each tier to a recommended outreach action with headcounts.
- A sortable **High-Risk Patient List** data-frame.
- An exportable **Full Risk Report CSV**.



Batch Analysis tab — live capture of the CSV upload area and instruction card.

10.3 AI Care Coordinator Tab (Structured Workflow + Chat)

This is the agentic surface. The left column mirrors the patient-profile form and contains three action buttons: **Analyze This Patient** (injects a structured prompt into the chat), **Run Full Care Workflow** (triggers the five-step structured pipeline), and **Generate & Download PDF Report**.

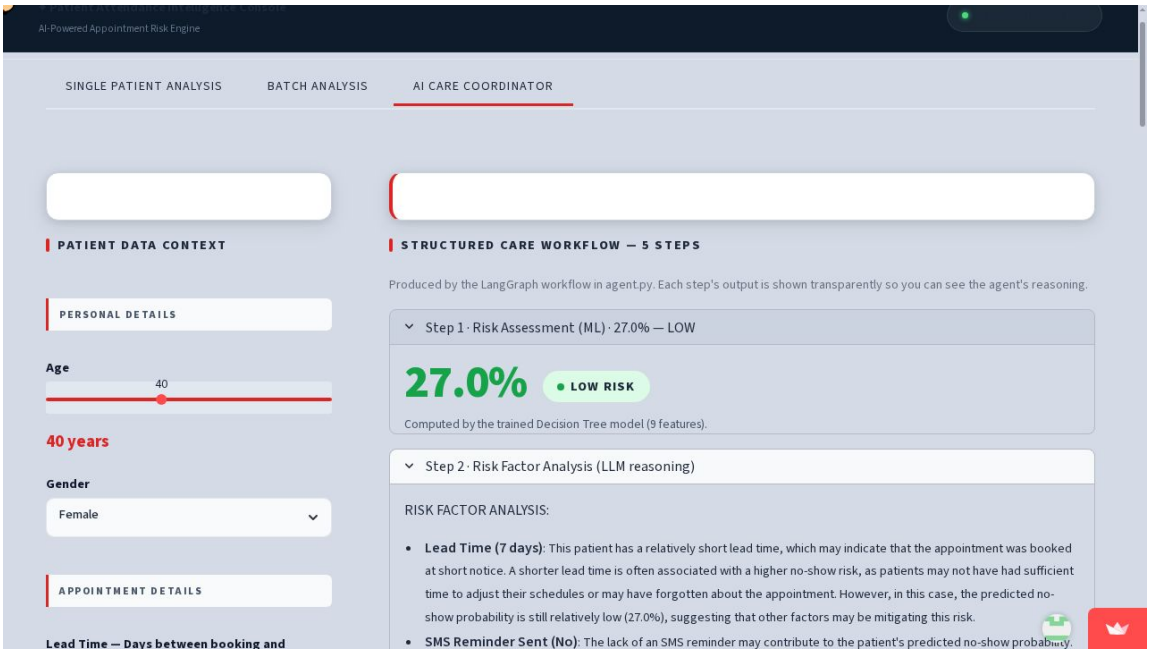
The right column displays either (a) the five-step structured workflow output when **Run Full Care Workflow** has been clicked, or (b) the ReAct chat — or both stacked, with the structured output above the chat.

The structured workflow renders as a single card titled **STRUCTURED CARE WORKFLOW — 5 STEPS** containing five expandable sections:

- **Step 1 — Risk Assessment (ML)** — large colour-coded percentage and tier badge.
- **Step 2 — Risk Factor Analysis (LLM reasoning)** — the patient-specific factor analysis.
- **Step 3 — Guideline Retrieval (RAG)** — source-filename chips plus the retrieved excerpts with `[Source: <filename>]` prefixes.

- **Step 4 — Intervention Plan** — the grounded plan.
- **Step 5 — Final Structured Report** — the compiled care-coordination report.

A permanent *Operational & Ethical Notice* card is rendered beneath the five steps, and a **Clear Workflow Output** button resets the session state.



*Structured 5-step care workflow — live capture of the AI Care Coordinator tab after clicking **Run Full Care Workflow**, with Step 3 (Guideline Retrieval) expanded.*

10.4 Deployment Targets

Platform	Configuration
Local	`python-dotenv` for `GROQ_API_KEY`; `streamlit run app.py` on `localhost:8501`.
Streamlit Cloud	`st.secrets` for `GROQ_API_KEY`; pre-built `chroma_db/` committed to repo; auto-redeploy on every push to `main`.
Hugging Face Spaces	Optional alternative — YAML header in `README.md` configures the Space; secrets added in Space Settings.

Live Demo: Patient Attendance Intelligence Console

11. Practical Implications

The developed system can assist healthcare operations teams at multiple levels:

- **Automated scoring.** The Decision Tree model provides consistent, unbiased probability scores that remove subjective variability from front-desk triage of reminder lists.

- **Explainable recommendations.** Feature-importance-driven narratives and LLM-generated factor analyses translate model outputs into language that clinic staff and care coordinators can understand and act on.
- **Policy compliance.** The RAG copilot retrieves applicable hospital-operations guidelines in real time, helping staff verify that outreach decisions align with current internal protocols without manually searching through lengthy policy documents.
- **Scenario planning.** The what-if simulation allows operational staff to advise patients on specific changes (earlier appointment slots, confirmed SMS opt-ins) that would shift their risk classification.
- **Audit traceability.** The downloadable structured PDF report captures the full decision rationale — risk score, factor analysis, retrieved guideline sources, intervention plan — for compliance records and internal review.

12. Limitations and Future Work

Minority-class recall. The baseline Decision Tree is trained without class-weighting and consequently catches only $\sim$7.4% of actual no-shows. Class-weighted training, probability-threshold calibration, or a gradient-boosted ensemble (XGBoost / LightGBM with `scale_pos_weight`) would tighten this materially. The tier-mapping threshold in the agentic layer partially compensates by converting raw probability into ordinal bins rather than using the default 0.5 cut-off, but improving the underlying classifier is the primary future-work item.

LLM capacity. The free-tier `llama-3.1-8b-instant` model occasionally truncates long structured outputs, particularly during the Step-5 report-generation stage. A larger model (Llama 3.1 70B, or a paid-tier completion) would smooth this out at higher cost per query.

Corpus coverage. Five markdown documents are sufficient to demonstrate RAG end-to-end but are unlikely to cover every hospital-specific operational policy. Future deployments should index the full operations manual and add a *freshness* check that flags stale guidelines for periodic human review.

Retrieval evaluation. We evaluate retrieval qualitatively through the live-verification case study. A formal evaluation with a held-out query-to-chunk labelled set (hit@k, nDCG) is future work.

Missing operational features. The original capstone brief mentions *Department* and *Day/-Time* as intended features. The Kaggle V2 dataset does not expose department; day-of-week could be engineered from `AppointmentDay` and would be a cheap future addition, as would an encoded hour-of-day bucket.

13. Conclusion

This project successfully demonstrates the application of machine learning for clinical appointment no-show prediction and extends it into a full-scale Agentic AI care-coordination platform. The Decision Tree classifier effectively captures the complex, non-linear relationships between patient attributes and attendance behaviour on a heavily imbalanced dataset, and the engineered `AwaitingTime` and `Num_App_Missed` features prove to be the dominant signals — consistent with the published healthcare-operations literature.

Beyond the ML model, the LangGraph agentic workflow, five-layer guardrail architecture, Chro-

maDB RAG pipeline, and hardened LLM system prompt together produce a system that is simultaneously accurate, explainable, policy-grounded, and robust against the classic RAG failure mode of fluent hallucination under empty retrieval. The Streamlit dashboard makes all of this accessible to non-technical clinic staff through an intuitive, real-time interface.

The design philosophy throughout was one of **controlled AI autonomy**: the LLM components are given latitude to generate helpful, nuanced narratives and regulatory analysis, but they operate within strict deterministic boundaries — the risk tier is fixed by the ML model, every policy statement must be tagged with a verifiable source filename, and every structured output carries an operational and ethical disclaimer. This architecture reflects the emerging best practice in agentic AI system design and is well-suited for high-stakes operational applications where explainability and auditability are non-negotiable.

14. Team Contributions

Team Member	Contributions
Ansh Badkur	Handled initial project ideation, problem definition, and dataset sourcing from Kaggle. Built the complete Milestone-1 ML layer including data cleaning, invalid-age filtering, categorical encoding for `Handcap` and `Gender`, one-hot transformations, feature scaling with `StandardScaler`, and the two engineered features `AwaitingTime` and `Num_App_Missed`. Performed model selection between Logistic Regression and the Decision Tree, carried out hyperparameter tuning (`max_depth`$= 10$), and produced the Milestone-1 evaluation metrics. Authored the base Streamlit application layout, custom CSS theme, and the Single Patient Analysis and Batch Analysis tabs including the risk-distribution Plotly chart and the operational-actions panel. Scaffolded the ReAct chat agent (`chatbot_agent.py`) with the `predict_noshow` and `search_guidelines` tools, including the initial system prompt. Wrote the fpdf2-based `pdf_generator.py` module for downloadable care-coordination reports. Performed the initial deployment of the application to Streamlit Community Cloud.

Team Member	Contributions (cont'd)
Praanshu Ranjan	Diagnosed and fixed the critical production failure where the ChromaDB vector store was excluded from the Git repository by `.gitignore`, which caused the deployed RAG layer to return zero matches and the agent to hallucinate invented hospital policies despite the corpus being present on local machines. Rebuilt the 40-chunk vector store from the five markdown source documents and committed the populated `chroma_db/` directory to the repository so the hosted container ships with a populated knowledge base on first boot. Hardened the ReAct agent's system prompt and tool docstrings from three generic sentences into a codified behavioural contract that requires every policy claim to carry a `[Source: <filename>]` tag, forbids fabricated policies when retrieval is empty, mandates that the exact risk percentage be surfaced alongside the tier, and enforces the operational-and-ethical disclaimer on every structured output. Integrated the previously-unused five-node LangGraph structured workflow (`agent.py`) into the AI Care Coordinator tab by adding the **Run Full Care Workflow** button, a transparent per-step UI with five expandable cards (Risk Assessment / Risk Reasoning / Guideline Retrieval with source chips / Intervention Plan / Final Report), a permanent operational-and-ethical notice, and a **Clear Workflow Output** button for state reset. Fixed a visual rendering bug on the **Run Full Care Workflow** button caused by an interaction between Streamlit's `help=` tooltip wrapper and the project's custom CSS selectors. Rewrote the project README from scratch for the end-semester submission, including the live-application badge, Mermaid architecture diagrams, environment and deployment instructions, team credits, and the ethical disclaimer. Authored this end-semester project report. Produced the final five-minute demonstration video. Verified the post-fix live application end-to-end using automated browser testing, confirming that the deployed agent now cites real source filenames from the hospital guidelines corpus rather than fabricating policy content.